

那么栈空间究竟有多大呢？这和操作系统相关。在 Linux 中，栈大小是由系统命令 `ulimit` 指定的，例如，`ulimit -a` 显示当前栈大小，而 `ulimit -s 32768` 将把栈大小指定为 32MB。但在 Windows 中，栈大小是储存在可执行文件中的。使用 `gcc` 可以这样指定可执行文件的栈大小：`gcc -Wl,--stack=16777216①`，这样栈大小就变为 16MB。

提示 4-21：在 Linux 中，栈大小并没有储存在可执行程序中，只能用 `ulimit` 命令修改；在 Windows 中，栈大小储存在可执行程序中，用 `gcc` 编译时可以通过 `-Wl,--stack=<byte count>` 指定。

聪明的读者，现在你能理解为什么在介绍数组时，建议“把较大的数组放在 `main` 函数外”了吗？别忘了，局部变量也是放在堆栈段的。栈溢出不一定是递归调用太多，也可能是局部变量太大。只要总大小超过了允许的范围，就会产生栈溢出。

4.4 竞赛题目选讲

从技术上讲，不用函数和递归也可以写出所有程序^②。但是从实用的角度来讲，函数和递归能帮我们大忙。人毕竟不是机器，代码的可读性和可维护性是相当重要的。很多初学者渴望学习到更好的调试技巧，但在此之前，笔者却总是建议他们先学习如何更好地写程序。如果方法得当，不仅能更快地写出更短的程序，而且调试起来也更轻松，隐含的错误也会更少。本节的题目并不涉及新的知识点，但在程序组织和调试技巧上会给读者一些新的启示。

例题 4-2 刽子手游戏 (Hangman Judge, UVa 489)

刽子手游戏其实是一款猜单词游戏，如图 4-1 所示。游戏规则是这样的：计算机想一个单词让你猜，你每次可以猜一个字母。如果单词里有那个字母，所有该字母会显示出来；如果没有那个字母，则计算机会在一幅“刽子手”画上填一笔。这幅画一共需要 7 笔就能完成，因此你最多只能错 6 次。注意，猜一个已经猜过的字母也算错。

在本题中，你的任务是编写一个“裁判”程序，输入单词和玩家的猜测，判断玩家赢了 (You win.)、输了 (You lose.) 还是放弃了 (You chickened out.)。每组数据包含 3 行，第 1 行是游戏编号 (-1 为输入结束标记)，第 2 行是计算机想的单词，第 3 行是玩家的猜测。后两行保证只含小写字母。

样例输入：

```
1
cheese
```

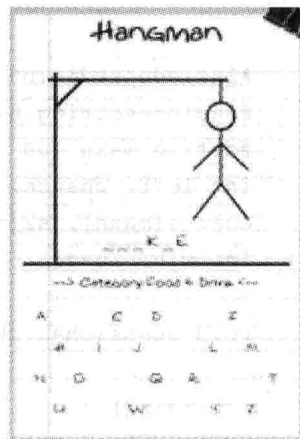


图 4-1 刽子手游戏

^① 实际上，栈大小是由连接程序 `ld` 指定的。`gcc` 编译参数 `-Wl` 的作用正是把其后的参数 (`--stack=<size>`) 传递给 `ld`。

^② 这里没有“几乎”二字。函数和递归均可以用其他内容替代。

```
cheese
2
cheese
abcdefg
3
cheese
abcdefghijkl
-1
```

样例输出：

```
Round 1
You win.
Round 2
You chickened out.
Round 3
You lose.
```

【分析】

一般而言，程序不是直接从第一行开始写到最后一行结束，而是遵循两种常见的顺序之一：自顶向下和自底向上。什么叫自顶向下呢？简单地说，就是先写框架，再写细节。实际上，之前已经用过这个方法了，就是先写“伪代码”，然后转化成实际的代码。有了“函数”这个工具之后，可以更好地贯彻这个方法：先写主程序，包括对函数的调用，再实现函数本身。自底向上和这个顺序相反，是先写函数，再写主程序。对于编写复杂软件来说，自底向下的构建方式有它独特的优势^①。但在算法竞赛中，这样做的选手并不多见^②。

程序 4-11 刽子手游戏——程序框架

```
#include<stdio.h>
#include<string.h>
#define maxn 100
int left, chance;           //还需要猜 left 个位置，错 chance 次之后就会输
char s[maxn], s2[maxn];    //答案是字符串 s，玩家猜的字母序列是 s2
int win, lose;              //win=1 表示已经赢了；lose=1 表示已经输了

void guess(char ch) { ... }

int main() {
    int rnd;
    while(scanf("%d%s", &rnd, s, s2) == 3 && rnd != -1) {
        printf("Round %d\n", rnd);
        win = lose = 0;           //求解一组新数据之前要初始化
```

^① 有兴趣的读者可以翻阅 Paul Graham 的经典著作《On Lisp》。

^② 注意：这里讨论的是编写代码的顺序。在测试时，先测试工具函数的方式非常常用。

```

left = strlen(s);
chance = 7;
for(int i = 0; i < strlen(s2); i++) {
    guess(s2[i]);           //猜一个字母
    if(win || lose) break;  //检查状态
}
//根据结果进行输出
if(win) printf("You win.\n");
else if(lose) printf("You lose.\n");
else printf("You chickened out.\n");
}
return 0;
}

```

有一些细节需要说明。

一是变量名的选取。那个 `rnd` 本应叫 `round`，但是有一个库函数也叫 `round`，所以改名叫 `rnd` 了。当然，改成 `Round` 也可以，因为 C 语言的标识符是区分大小写的。这里改成 `rnd` 只是个人习惯。毕竟这个代码很短，而且 `rnd` 这个变量的作用域很小，很容易搞清楚它的含义。在第 5 章学习完 STL 之后，这种“被用过的常用名字”还会增加，例如 `count`、`min`、`max` 等都是 STL 已经使用的名字，程序中最好避开它们。

二是变量的使用。全局变量本应该尽量少用，但是对于本题来说，需要维护的内容比较多，例如，是否赢了，是否输了，以及剩余的机会数等。如果不用全局变量，则它们都需要传递给函数 `guess`。更麻烦的是，其中有些参数还需要被 `guess` 修改，只能传指针，但这会让代码变“丑^①”。所以笔者最终选择了使用全局变量。读者完全可以对此持不同看法，刚才的文字只是想说明：变量和函数调用方式的设计是一个需要思考的问题。如果设计出的方案还未写出便觉得别扭，恐怕写出来的程序会既不优美，也不好调试，甚至容易隐藏 bug。

下一步是实现 `guess` 函数。在编写这个函数时，可能会注意到一个问题：题目中说了猜过的字母再猜一次算错，可是似乎并没有保存哪些字母已经猜过。一个解决方案是在程序框架中增加一个字符数组 `int guessed[256]`，让 `guessed[ch]` 标识字母 `ch` 是否已经猜过。但其实还有一个更简单的方法，就是将猜对的字符改成空格，像这样：

程序 4-12 刽子手游戏——`guess` 函数

```

void guess(char ch) {
    int bad = 1;
    for(int i = 0; i < strlen(s); i++)
        if(s[i] == ch) { left--; s[i] = ' '; bad = 0; }
    if(bad) --chance;
    if(!chance) lose = 1;
    if(!left) win = 1;
}

```

^① 当然，这是笔者的主观看法。有些人觉得充满指针的代码很优美。

这样, 程序就完整了。如何调试呢? 每猜完一个字母之后打印出 `s`、`left`、`chance` 等重要变量的值, 很容易就能发现程序出错的位置, 读者不妨一试。另一方面, 如果刚才加上了 `guessed` 数组, 每次打印的调试信息就会多出这样一个庞大的数组, 不仅数据多, 而且不直观, 会给调试带来麻烦。一般来说, 减少变量的个数对于编程和调试都会有帮助。

例题 4-3 救济金发放 (The Dole Queue, UVa 133)

$n(n < 20)$ 个人站成一圈, 逆时针编号为 $1 \sim n$ 。有两个官员, A 从 1 开始逆时针数, B 从 n 开始顺时针数。在每一轮中, 官员 A 数 k 个就停下来, 官员 B 数 m 个就停下来 (注意有可能两个官员停在同一个人上)。接下来被官员选中的人 (1 个或者 2 个) 离开队伍。

输入 n, k, m 输出每轮里被选中的人的编号 (如果有两个人, 先输出被 A 选中的)。例如, $n=10, k=4, m=3$, 输出为 4 8, 9 5, 3 1, 2 6, 10, 7。注意: 输出的每个数应当恰好占 3 列。

【分析】

仍然采用自顶向下的方法编写程序。用一个大小为 0 的数组表示人站成的圈。为了避免人走之后移动数组元素, 用 0 表示离开队伍的人, 数数时跳过即可。主程序如下:

```
#include<stdio.h>
#define maxn 25
int n, k, m, a[maxn];

//逆时针走 t 步, 步长是 d (-1 表示顺时针走), 返回新位置
int go(int p, int d, int t) { ... }

int main() {
    while(scanf("%d%d%d", &n, &k, &m) == 3 && n) {
        for(int i = 1; i <= n; i++) a[i] = i;
        int left = n; //剩下的人数
        int p1 = n, p2 = 1;
        while(left) {
            p1 = go(p1, 1, k);
            p2 = go(p2, -1, m);
            printf("%3d", p1); left--;
            if(p2 != p1) { printf("%3d", p2); left--; }
            a[p1] = a[p2] = 0;
            if(left) printf(",");
        }
        printf("\n");
    }
    return 0;
}
```

注意 `go` 这个函数。当然也可以写两个函数: 逆时针 `go` 和顺时针 `go`, 但是仔细思考后发现这两个函数可以合并: 逆时针和顺时针数数的唯一区别只是下标是加 1 还是减 1。把这

个+1/-1抽象为“步长”参数，就可以把两个go统一了。代码如下：

```
int go(int p, int d, int t) {
    while(t--) {
        do { p = (p+d+n-1) % n + 1; } while(a[p] == 0); //走到下一个非0数字
    }
    return p;
}
```

例题 4-4 信息解码 (Message Decoding, ACM/ICPC World Finals 1991, UVa 213)

考虑下面的 01 串序列：

0, 00, 01, 10, 000, 001, 010, 011, 100, 101, 110, 0000, 0001, ..., 1101, 1110, 00000, ...

首先是长度为 1 的串，然后是长度为 2 的串，依此类推。如果看成二进制，相同长度的后一个串等于前一个串加 1。注意上述序列中不存在全为 1 的串。

你的任务是编写一个解码程序。首先输入一个编码头（例如 AB#TANCnrtXc），则上述序列的每个串依次对应编码头的每个字符。例如，0 对应 A，00 对应 B，01 对应 #，...，110 对应 X，0000 对应 c。接下来是编码文本（可能由多行组成，你应当把它们拼成一个长长的 01 串）。编码文本由多个小节组成，每个小节的前 3 个数字代表小节中每个编码的长度（用二进制表示，例如 010 代表长度为 2），然后是各个字符的编码，以全 1 结束（例如，编码长度为 2 的小节以 11 结束）。编码文本以编码长度为 000 的小节结束。

例如，编码头为 \$**\，编码文本为 0100000101101100011100101000，应这样解码：010(编码长度为 2)00(#)00(#)10(*)11(小节结束)011(编码长度为 3)000(\)111(小节结束)001(编码长度为 1)0(\$)1(小节结束)000(编码结束)。

【分析】

还记得二进制吗？如果不记得，请重新翻阅第 3 章的最后部分。有了二进制，就不必以字符串的形式保存这一大串编码了，只需把编码理解成二进制，用(len, value)这个二元组来表示一个编码，其中 len 是编码长度，value 是编码对应的十进制值。如果用 codes[len][value] 保存这个编码所对应的字符，则主程序看上去应该是这个样子的。

```
#include<stdio.h>
#include<string.h> //使用 memset
int readchar() { ... }
int readint(int c) { ... }
```

```
int code[8][1<<8];
```

```
int readcodes() { ... }
```

```
int main() {
```

```
    while(readcodes()) { //无法读取更多编码头时退出
```

```
        //printcodes();
```

```
        for(;;) {
```

```

int len = readint(3);
if(len == 0) break;
//printf("len=%d\n", len);
for(;;) {
    int v = readint(len);
    //printf("v=%d\n", v);
    if(v == (1 << len)-1) break;
    putchar(code[len][v]);
}
putchar('\n');
}
return 0;
}

```

主程序里接连使用了两个还没有介绍的函数：`readcodes` 和 `readint`。前者用来读取编码，后者读取 `c` 位二进制字符（即 0 和 1），并转化为十进制整数。

本题的调试方法也很有代表性。上面的代码中已经包含了几条注释掉的 `printf` 语句，用于打印出一些关键变量的值。如果程序的输出不是想要的结果，题目中的举例就派上用场了：只需把举例中的解释和程序输出的中间结果一一对照，就能知道问题出在哪里。

编写 `readint` 时会遇到同一个问题：如何处理“编码文本可以由多行组成”这个问题？方法有很多种，笔者的方案是再编写一个“跨行读字符”的函数 `readchar`。

```

int readchar() {
    for(;;) {
        int ch = getchar();
        if(ch != '\n' && ch != '\r') return ch; //一直读到非换行符为止
    }
}

int readint(int c) {
    int v = 0;
    while(c--) v = v * 2 + readchar() - '0';
    return v;
}

```

下面是函数 `readcodes`。首先使用 `memset` 清空数组（这是个好习惯。还记得之前讲过的多数据题目的常见错误吗？），编码头自身占一行，所以应该用 `readchar` 读取第一个字符，而用普通的 `getchar` 读取剩下的字符，直到 `\n`。这样做，代码比较简单，但有些读者可能会觉得有些别扭。没关系，你完全可以使用另外一套自己觉得更清晰的方法。

```

int readcodes() {
    memset(code, 0, sizeof(code)); //清空数组
}

```

```

code[1][0] = readchar(); //直接调到下一行开始读取。如果输入已经结束，会读到 EOF
for(int len = 2; len <= 7; len++) {
    for(int i = 0; i < (1<<len)-1; i++) {
        int ch = getchar();
        if(ch == EOF) return 0;
        if(ch == '\n' || ch == '\r') return 1;
        code[len][i] = ch;
    }
}
return 1;

```

最后是前面提到的 `printcodes` 函数。这个函数对于解题来说不是必需的，但对于调试却是有用的。

```

void printcodes() {
    for(int len = 1; len <= 7; len++)
        for(int i = 0; i < (1<<len)-1; i++) {
            if(code[len][i] == 0) return;
            printf("code[%d][%d] = %c\n", len, i, code[len][i]);
        }
}

```

由于每次读取编码头时把 `codes` 数组清空了，所以只要遇到字符为 0 的情况，就表示编码头已经结束。

例题 4-5 追踪电子表格中的单元格 (Spreadsheet Tracking, ACM/ICPC World Finals 1997, UVa512)

有一个 r 行 c 列 ($1 \leq r, c \leq 50$) 的电子表格，行从上到下编号为 $1 \sim r$ ，列从左到右编号为 $1 \sim c$ 。如图 4-2 (a) 所示，如果先删除第 1、5 行，然后删除第 3, 6, 7, 9 列，结果如图 4-2 (b) 所示。

	A	B	C	D	E	F	G	H	I
1	22	55	66	77	88	99	10	12	14
2	2	24	6	8	22	12	14	16	18
3	18	19	20	21	22	23	24	25	26
4	24	25	26	67	22	69	70	71	77
5	68	78	79	80	22	25	28	29	30
6	16	12	11	10	22	56	57	58	59
7	33	34	35	36	22	38	39	40	41

(a)

	A	B	C	D	E
1	2	24	8	22	16
2	18	19	21	22	25
3	24	25	67	22	71
4	16	12	10	22	58
5	33	34	36	22	40

(b)

图 4-2 删除行、列

接下来在第 2、3、5 行前各插入一个空行，然后在第 3 列前插入一个空列，会得到如图 4-3 所示结果。

	A	B	C	D	E	F
1	2	24		8	22	16
2						
3	18	19		21	22	25
4						
5	24	25		67	22	71
6	16	12		10	22	58
7						
8	33	34		36	22	40

图 4-3 插入行、列

你的任务是模拟这样的 n 个操作。具体来说一共有 5 种操作:

□ EX $r_1\ c_1\ r_2\ c_2$ 交换单元格 $(r_1, c_1), (r_2, c_2)$ 。

□ <command> A $x_1\ x_2\ \dots\ x_A$ 插入或删除 A 行或列 (DC-删除列, DR-删除行, IC-插入列, IR-插入行, $1 \leq A \leq 10$)。

在插入/删除指令后, 各个 x 值不同, 且顺序任意。接下来是 q 个查询, 每个查询格式为 “rc”, 表示查询原始表格的单元格 (r, c) 。对于每个查询, 输出操作执行完后该单元格的新位置。输入保证在任意时刻行列数均不超过 50。

【分析】

最直接的思路就是首先模拟操作, 算出最后的电子表格, 然后在每次查询时直接在电子表格中找到所求的单元格。为了锻炼读者的代码阅读能力, 此处不对代码进行任何解释:

```
#include<stdio.h>
#include<string.h>
#define maxd 100
#define BIG 10000
int r, c, n, d[maxd][maxd], d2[maxd][maxd], ans[maxd][maxd], cols[maxd];

void copy(char type, int p, int q) {
    if(type == 'R') {
        for(int i = 1; i <= c; i++)
            d[p][i] = d2[q][i];
    } else {
        for(int i = 1; i <= r; i++)
            d[i][p] = d2[i][q];
    }
}

void del(char type) {
    memcpy(d2, d, sizeof(d));
    int cnt = type == 'R' ? r : c, cnt2 = 0;
    for(int i = 1; i <= cnt; i++) {
        if(!cols[i]) copy(type, ++cnt2, i);
    }
}
```



```

    if(type == 'R') r = cnt2; else c = cnt2;
}

void ins(char type) {
    memcpy(d2, d, sizeof(d));
    int cnt = type == 'R' ? r : c, cnt2 = 0;
    for(int i = 1; i <= cnt; i++) {
        if(cols[i]) copy(type, ++cnt2, 0);
        copy(type, ++cnt2, i);
    }
    if(type == 'R') r = cnt2; else c = cnt2;
}

int main() {
    int r1, c1, r2, c2, q, kase = 0;
    char cmd[10];
    memset(d, 0, sizeof(d));
    while(scanf("%d%d%d", &r, &c, &n) == 3 && r) {
        int r0 = r, c0 = c;
        for(int i = 1; i <= r; i++)
            for(int j = 1; j <= c; j++)
                d[i][j] = i*BIG + j;
        while(n--) {
            scanf("%s", cmd);
            if(cmd[0] == 'E') {
                scanf("%d%d%d%d", &r1, &c1, &r2, &c2);
                int t = d[r1][c1]; d[r1][c1] = d[r2][c2]; d[r2][c2] = t;
            } else {
                int a, x;
                scanf("%d", &a);
                memset(cols, 0, sizeof(cols));
                for(int i = 0; i < a; i++) { scanf("%d", &x); cols[x] = 1; }
                if(cmd[0] == 'D') del(cmd[1]); else ins(cmd[1]);
            }
        }
        memset(ans, 0, sizeof(ans));
        for(int i = 1; i <= r; i++)
            for(int j = 1; j <= c; j++) {
                ans[d[i][j]/BIG][d[i][j]%BIG] = i*BIG+j;
            }
        if(kase > 0) printf("\n");
        printf("Spreadsheet #%d\n", ++kase);
    }
}

```

```
scanf("%d", &q);
while(q--) {
    scanf("%d%d", &r1, &c1);
    printf("Cell data in (%d,%d) ", r1, c1);
    if(ans[r1][c1] == 0) printf("GONE\n");
    else printf("moved to (%d,%d)\n", ans[r1][c1]/BIG, ans[r1][c1]%BIG);
}
}
return 0;
}
```

另一个思路是将所有操作保存, 然后对于每个查询重新执行每个操作, 但不需要计算整个电子表格的变化, 而只需关注所查询的单元格的位置变化。对于题目给定的规模来说, 这个方法不仅更好写, 而且效率更高。代码如下:

```
#include<stdio.h>
#include<string.h>
#define maxd 10000

struct Command {
    char c[5];
    int r1, c1, r2, c2;
    int a, x[20];
} cmd[maxd];

int r, c, n;

int simulate(int* r0, int* c0) {
    for(int i = 0; i < n; i++) {
        if(cmd[i].c[0] == 'E') {
            if(cmd[i].r1 == *r0 && cmd[i].c1 == *c0) { *r0 = cmd[i].r2; *c0 = cmd[i].c2; }
            else if(cmd[i].r2 == *r0 && cmd[i].c2 == *c0) { *r0 = cmd[i].r1; *c0 = cmd[i].c1; }
        } else {
            int dr = 0, dc = 0;
            for(int j = 0; j < cmd[i].a; j++) {
                int x = cmd[i].x[j];
                if(cmd[i].c[0] == 'I') {
                    if(cmd[i].c[1] == 'R' && x <= *r0) dr++;
                    if(cmd[i].c[1] == 'C' && x <= *c0) dc++;
                }
            }
            else {
                if(cmd[i].c[1] == 'R' && x == *r0) return 0;
            }
        }
    }
}
```

```

        if(cmd[i].c[1] == 'C' && x == *c0) return 0;
        if(cmd[i].c[1] == 'R' && x < *r0) dr--;
        if(cmd[i].c[1] == 'C' && x < *c0) dc--;
    }
}
*r0 += dr; *c0 += dc;
}
return 1;
}

int main() {
    int r0, c0, q, kase = 0;
    while(scanf("%d%d%d", &r, &c, &n) == 3 && r) {
        for(int i = 0; i < n; i++) {
            scanf("%s", cmd[i].c);
            if(cmd[i].c[0] == 'E') {
                scanf("%d%d%d%d", &cmd[i].r1, &cmd[i].c1, &cmd[i].r2, &cmd[i].c2);
            } else {
                scanf("%d", &cmd[i].a);
                for(int j = 0; j < cmd[i].a; j++) scanf("%d", &cmd[i].x[j]);
            }
        }
        if(kase > 0) printf("\n");
        printf("Spreadsheet #%d\n", ++kase);

        scanf("%d", &q);
        while(q--) {
            scanf("%d%d", &r0, &c0);
            printf("Cell data in (%d,%d) ", r0, c0);
            if(!simulate(&r0, &c0)) printf("GONE\n");
            else printf("moved to (%d,%d)\n", r0, c0);
        }
    }
    return 0;
}

```

有没有觉得 `simulate` 函数不是特别自然？因为所有用到 `r0` 和 `c0` 的地方都要加上一个星号。幸运的是，C++语言中有另外一个语法，可以更自然地表达这种“需要被修改的参数”，详见第5章中的“引用”部分。

例题 4-6 师兄帮帮忙 (A Typical Homework (a.k.a Shi Xiong Bang Bang Mang), Rujia Liu's Present 5, UVa 12412)

(题目背景略，有兴趣的读者请自行阅读原题)

编写一个成绩管理系统（SPMS）。最多有 100 个学生，每个学生有如下属性。

- ☐ SID: 学生编号，包含 10 位数字。
- ☐ CID: 班级编号，为不超过 20 的正整数。
- ☐ 姓名: 不超过 10 的字母和数字组成，第一个字符为大写字母。名字中不能有空白字符。
- ☐ 4 门课程（语文、数学、英语、编程）成绩，均为不超过 100 的非负整数。

进入 SPMS 后，应显示主菜单：

```
Welcome to Student Performance Management System (SPMS).
```

```
1 - Add
2 - Remove
3 - Query
4 - Show ranking
5 - Show Statistics
0 - Exit
```

选择 1 之后，会出现添加学生记录的提示信息：

```
Please enter the SID, CID, name and four scores. Enter 0 to finish.
```

然后等待输入。本题保证输入总是合法的（不会有非法的 SID、CID，并且恰好有 4 个分数等），但可能会输入重复 SID。在这种情况下，需要输出一行提示：

```
Duplicated SID.
```

不过名字是可以重复的。你的程序应当不停地打印前述提示信息，直到用户输入单个 0。然后应当再次打印主菜单。

选择 2 之后，会出现如下提示信息：

```
Please enter SID or name. Enter 0 to finish.
```

然后等待输入，在数据库中删除能匹配上述 SID 或者名字的所有学生，并且打印如下信息（xx 可以等于 0）：

```
xx student(s) removed.
```

你的程序应当不停地打印前述提示信息，直到用户输入单个 0，然后再次打印主菜单。

选择 3 之后，会出现如下提示信息：

```
Please enter SID or name. Enter 0 to finish.
```

然后等待输入。如果数据库中没有能匹配上述 SID 或者名字的学生，什么都不要做；否则输出所有满足条件的学生，按照进入数据库的顺序排列。输出格式和添加的格式相同，但增加 3 列：年级排名（第一列）、总分和平均分（最后两列）。所有班级中总分最高的学生获得第 1 名，如果有两个学生并列第 2 名，则下一个学生的排名为 4（而非 3）。你的

程序应当不停地打印前述提示信息，直到用户输入单个 0。然后应当再次打印主菜单。

选择 4 之后，会出现如下提示信息：

```
Showing the ranklist hurts students' self-esteem. Don't do that.
```

然后自动返回主菜单。

选择 5 之后，会出现如下提示信息：

```
Please enter class ID, 0 for the whole statistics.
```

当用户输入班级 ID 之后（0 代表全年级），显示如下信息（注意，“及格”是指分数不小于 60 分）：

```
Chinese
```

```
Average Score: xx.xx
```

```
Number of passed students: xx
```

```
Number of failed students: xx
```

...（为了节约篇幅，此处省略了 Mathematics、English 和 Programming 的统计信息）

```
Overall:
```

```
Number of students who passed all subjects: xx
```

```
Number of students who passed 3 or more subjects: xx
```

```
Number of students who passed 2 or more subjects: xx
```

```
Number of students who passed 1 or more subjects: xx
```

```
Number of students who failed all subjects: xx
```

然后自动回到主菜单。

选择 0 之后，程序终止。注意，单科成绩和总分都应格式化为整数，但平均分应恰好保留两位小数。

提示：这个程序适合直接运行，用键盘与之交互，然后从屏幕中看到输出信息。但正因为如此，作为一道算法竞赛的题目，其输出看上去会比较乱。

【分析】

正如题目所说，这是一道很常见的“作业题”，在一些早期的大学编程教材中可以看到类似的问题（只是要求不一定有这么明确）。

因为要求比较多，可以沿用之前介绍过的“自顶向下，逐步求精”方法，先写出如下框架：

```
int main() {
    for(;;) {
        int choice;
        print_menu();
        scanf("%d", &choice);
        if(choice == 0) break;
```